

Differentially Private Formation Control: Privacy and Network Co-Design*

Calvin Hawkins[†]

Matthew Hale

Abstract

Privacy in multi-agent control is receiving increased attention, though often a networked system and privacy protections are designed separately, which can harm performance. Therefore, this paper presents a co-design framework for networks and private controllers, and we apply it to private formation control. Agents' state trajectories are protected using differential privacy, and we quantify its impact by bounding the steady-state error for private formations. Then, we analyze tradeoffs between privacy level, system performance, and connectedness of the network's communication topology. These tradeoffs are used to formulate a co-design optimization framework to jointly design agents' communication topology and their privacy levels. Simulation results illustrate the success of this framework.

Keywords: Network analysis and control, Privacy, Optimization

1 Introduction

Multi-agent systems require agents to share information to collaborate, and in some cases this information is sensitive. For example, self-driving cars may share location data to be routed to a destination. Location data and other streams can be revealing and thus require protections, though this data must still be useful for multi-agent coordination.

This type of privacy has recently been achieved using differential privacy [1], which has been applied to various control, estimation, and optimization problems [2, 3, 4, 5, 6, 7, 8]. Broadly speaking, differential privacy carefully introduces randomness in a system, e.g., by adding noise to conceal sensitive information. Differential privacy is appealing because it is immune to post-processing and robust to side information [9]. These properties mean that privacy guarantees are not compromised by performing operations on differentially private data, and that they are not weakened by much by an adversary with additional information about data-producing agents [10].

In this work we focus on distributed formation control, e.g., robots physically assembling into geometric shapes or non-physical agents maintaining relative state offsets. We develop a private decentralized formation control protocol in which each agent adds noise to keep its state trajectory private from other agents. We provide privacy guarantees over an agent's entire state trajectory, which ensures that an adversary is unlikely to learn much about the state trajectory of a system. Privacy noise of course impacts agents' formations, and we provide a closed-form for the steady-state error covariance that privacy induces in formations. We also bound the steady-state error itself, and both bounds are in terms of agents' privacy parameters.

Privacy is often only a post-hoc concern that is incorporated after a network and/or a controller is designed, which can harm performance; however, there also exist approaches that achieve privacy without sacrificing convergence [11]. Therefore, we leverage our steady-state analyses to formulate optimization problems that design a network topology and a differential privacy implementation together.

While other works have implemented other forms of privacy for trajectories in consensus problems, e.g., [12], to the best of our knowledge, we are the first to implement trajectory-level *differential* privacy in the setting of formation control. The formation control protocol does resemble the average consensus

*This work was supported by AFOSR under grant FA9550-19-1-0169, by ONR under grant N00014-21-1-2502, and by NSF CAREER grant 1943275. The authors are with the School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA, USA.

[†]Corresponding author. Emails: {chawkins64,mhale30}@gatech.edu

protocol, and existing work has applied differential privacy to initial states in consensus, e.g., [13, 14, 15] and others. This paper differs by implementing trajectory-level differential privacy [16], which conceals deviations from a nominal state trajectory, which can occur at any time and can occur across multiple timesteps, not just the initial time. There also exists work that considers differential privacy and *co-design* in a control setting [17], which considers initial state privacy and uses co-design to tune privacy noise and controller parameters, not the network topology. There also exists work on the co-design of control and communication, without addressing privacy [18].

This difference in privacy is necessary because we address a fundamentally different type of privacy need than initial-state privacy. To highlight the difference, suppose that a coalition of drones is assembling a formation, and, in the course of doing so, one drone takes a detour to photograph a location of interest, which it would like to conceal. This detour occurs after the initial time, and it cannot be concealed by privatizing its initial state. However, such a detour can be concealed by implementing privacy at the trajectory level, which is what we do.

Beyond differential privacy, other privacy-preserving approaches have been applied to networked control and estimation. For example, cryptographic approaches such as homomorphic encryption enable computation over a secure, shared channel at the cost of increased computational overhead [19]. Estimation-theoretic approaches based on information bounds have also been used to limit information disclosure by constraining inference accuracy [20]. In this work, we focus on differential privacy because it provides a formal, worst-case notion of privacy that is independent of adversary models and is well suited to decentralized control architectures.

While the privacy mechanism and some optimization formalisms employed in this work are standard, the problem formulation and analysis that connect privacy, network topology, and formation control performance are not. Our contributions therefore lie in making these connections analytically and developing a computational framework to apply them in practice.

In detail, the main contributions of this paper are as follows:

- An analytical characterization of the steady-state formation error under trajectory-level differential privacy as a function of the network topology
- A co-design formulation that jointly selects agent-level privacy parameters and communication weights under steady-state performance constraints
- A tractable solution approach based on a biconvex relaxation and alternate convex search which enables practical computation of co-designed networks.

The main analytical challenge in this work is to bound the steady-state formation error under trajectory-level differential privacy in a manner that explicitly captures the joint dependence on agent-level privacy parameters and the communication topology. This dependence is nonlinear and coupled, and therefore difficult to optimize directly. To overcome this difficulty we derive an explicit upper bound on steady-state error that isolates the dependence on privacy parameters and algebraic connectivity. The resulting bound is structured so that it can be embedded directly into a co-design optimization problem, enabling principled trade-offs between privacy strength, network connectivity, and formation performance.

A preliminary version of this work appeared in [21]. This paper’s new contributions are the co-design framework, the steady-state error analysis, new simulations, and proofs of all results. The rest of the paper is organized as follows. Section 2 gives background, and Section 3 provides problem statements. Then Section 4 analyzes private formation control, and Section 5 develops the co-design framework. Section 6 provides simulations, and Section 7 concludes.

Notation: $\mathbb{R}$ denotes the reals and $\mathbb{N}$ denotes the naturals. For $\ell \in \mathbb{N}$, let $[\ell] := \{1, \dots, \ell\}$. We use $I_a \in \mathbb{R}^{a \times a}$ for the identity matrix in a dimensions and $\mathbf{1}$ for the vector of all ones in $\mathbb{R}^N$. We use the norm $\|v\|_\infty = \max_{i \in [N]} |v_i|$.

2 Background and Preliminaries

Here we review graph theory and differential privacy.

2.1 Graph Theory Background

A graph $\mathcal{G} = (V, E)$ is defined over a set of N nodes indexed over $V = \{1, \dots, N\}$ and a set of edges $E \subseteq V \times V$. The pair $(i, j) \in E$ if nodes i and j share a connection and $(i, j) \notin E$ if they do not. This paper considers undirected, weighted, simple graphs. Undirectedness means that an edge $(i, j) \in E$ is not distinguished from $(j, i) \in E$. Simplicity means that $(i, i) \notin E$ for all $i \in V$. Weightedness means that the edge $(i, j) \in E$ has a weight $w_{ij} = w_{ji} > 0$.

Definition 1 (Connected Graph). *A graph $\mathcal{G}$ is connected if, for all $i, j \in \{1, \dots, N\}$, $i \neq j$, there is a sequence of edges one can traverse from node i to node j .*

We assume the following in the rest of the paper.

Assumption 1. *Agents communicate over a connected, weighted, simple, undirected graph $\mathcal{G} = (V, E)$ on N nodes.*

This paper uses the weighted graph Laplacian, which is defined with weighted adjacency and degree matrices. The weighted adjacency matrix $A(\mathcal{G}) \in \mathbb{R}^{N \times N}$ of $\mathcal{G}$ is given by

$$A(\mathcal{G})_{ij} = \begin{cases} w_{ij} & (i, j) \in E \\ 0 & \text{otherwise} \end{cases}.$$

For undirected $\mathcal{G}$, $A(\mathcal{G})$ is symmetric. The degree matrix is $D(\mathcal{G}) = \text{diag}(d_1, \dots, d_N)$, and the Laplacian is $L(\mathcal{G}) = D(\mathcal{G}) - A(\mathcal{G})$. Agent i 's neighborhood set N_i is the set of all agents that agent i communicates with, i.e., $N_i = \{j \mid (i, j) \in E\}$. The weighted degree of node i is $d_i = \sum_{j \in N_i} w_{ij}$.

Let $\lambda_k(\cdot)$ be the k^{th} smallest eigenvalue of a symmetric matrix. Then for all graphs $0 = \lambda_1(L(\mathcal{G})) \leq \lambda_2(L(\mathcal{G})) \leq \dots \leq \lambda_N(L(\mathcal{G}))$.

Definition 2 (Algebraic Connectivity [22]). *The algebraic connectivity of a graph $\mathcal{G}$ is the second smallest eigenvalue of $L(\mathcal{G})$, and $\mathcal{G}$ is connected if and only if $\lambda_2(L(\mathcal{G})) > 0$.*

2.2 Differential Privacy Background

This section briefly reviews differential privacy. More detail can be found in [16, 23]. The goal of differential privacy is to make similar pieces of data appear approximately indistinguishable, and it is immune to post-processing [23]. That is, arbitrary post-hoc computations on private data do not harm differential privacy, including filtering [16, 24].

We apply differential privacy to agents' state trajectories, and we consider vector-valued trajectories of the form $Z = (Z(1), Z(2), \dots, Z(k), \dots)$, where $Z(k) \in \mathbb{R}^d$ for all k . The ℓ_2 norm of Z is defined as $\|Z\|_{\ell_2} = (\sum_{k=1}^{\infty} \|Z(k)\|_2^2)^{\frac{1}{2}}$, where $\|\cdot\|_2$ is the ordinary 2-norm on $\mathbb{R}^d$. We privatize trajectories in $\ell_{\infty}^d = \{Z \mid \|Z(k)\|_{\infty} < \infty \text{ for all } k\}$, namely the set of vector-valued trajectories whose entries are all finite. The set ℓ_{∞}^d contains trajectories that do not converge, which admits a wide variety of trajectories seen in control systems.

We consider a network of N agents, where agent i 's state trajectory is denoted by x_i . Its k^{th} element is denoted $x_i(k) \in \mathbb{R}^d$ for $d \in \mathbb{N}$, and agent i 's state trajectory belongs to ℓ_{∞}^d . In this work, agents privatize their state trajectories before they are shared, which contrasts with works that privatize collections of trajectories. The approach we take is called *input perturbation* [16], and it amounts to privatizing data, then using it in some computation. This is in place of *output perturbation*, in which one performs some computation with sensitive data and then privatizes its output. In principle, one may envision using output perturbation to privatize agents' state trajectories. However, the use of output perturbation requires aggregating agents' states in one place, and such an aggregator is typically absent in formation control.

Indeed, if an aggregator were available, then all agents could send their initial states to that aggregator, it could execute the formation control protocol in a centralized way, and then the aggregator could send each agent their final state to travel to. The decentralized formation control protocol is thus only meaningful in the absence of an aggregator, and we therefore choose a privacy implementation that fits with the lack of an aggregator, which is input perturbation.

This setup has the advantages of allowing each agent to calibrate its own privacy parameters, and it also ensures that agents' information is privatized before it is ever shared. A known challenge can be that more noise is often required for input perturbation privacy than in alternative setups.

The goal of differential privacy is to make “similar” pieces of data approximately indistinguishable, and an adjacency relation quantifies when pieces of data are “similar.” For input perturbation, adjacency is defined for single agents.

Definition 3 (Adjacency). *Fix an adjacency parameter $b_i > 0$ for agent i . Then $\text{Adj}_{b_i} : \ell_\infty^d \times \ell_\infty^d \rightarrow \{0, 1\}$ is defined as*

$$\text{Adj}_{b_i}(v_i, r_i) = \begin{cases} 1 & \|v_i - r_i\|_{\ell_2} \leq b_i \\ 0 & \text{otherwise.} \end{cases}$$

In words, two state trajectories that agent i could produce are adjacent if and only if the ℓ_2 -norm of their difference is upper bounded by b_i . This means that every state trajectory within ℓ_2 distance b_i from agent i 's actual state trajectory must be made approximately indistinguishable from it. For example, agent i may take a detour of size b_i from the nominal trajectory that it would follow to get into formation. This detour can occur at any time, and the adjacency relation in Definition 3 requires that our privacy implementation conceal it, regardless of when it occurs. Existing approaches to initial-state privacy do not provide these protections.

While adjacency requires two state trajectories to have a finite difference between them, it does not require the two trajectories in question to converge. This is reflected in the fact that an agent's trajectories are contained in the space ℓ_∞^d , which indicates that trajectories consist of real-valued vectors in $\mathbb{R}^d$ that can take any finite values, and, in particular, they need not ever converge to any value.

Privacy is implemented by a “mechanism,” which is a randomized map that we define next. First, fix a probability space $(\Omega, \mathcal{F}, \mathbb{P})$. We consider outputs in ℓ_∞^d and use a σ -algebra over ℓ_∞^d , denoted $\mathcal{F}_\infty^d$ [25].

Definition 4 (Differential Privacy). *Let $\epsilon_i > 0$ and $\delta_i \in (0, \frac{1}{2})$ be given. Fix $b_i > 0$. A mechanism $M : \ell_\infty^d \times \Omega \rightarrow \ell_\infty^d$ is (ϵ_i, δ_i) -differentially private if, for all adjacent $x_i, x'_i \in \ell_\infty^d$, we have*

$$\mathbb{P}[M(x_i) \in S] \leq e^{\epsilon_i} \mathbb{P}[M(x'_i) \in S] + \delta_i$$

for all $S \in \mathcal{F}_\infty^d$.

Definition 4 is applied individually to each agent $i \in [N]$, i.e., each agent's released trajectory satisfies (ϵ_i, δ_i) -differential privacy with respect to Definition 3. To calibrate privacy, agent i selects privacy parameters ϵ_i and δ_i . Typically, $\epsilon_i \in [0.1, \ln 3]$ and $\delta_i \in [0, 0.05]$ for all i [16, 26, 10]. The value of δ_i can be regarded as allowing a small probability that the ϵ_i -differential privacy guarantee for agent i is violated, while ϵ_i controls the strength of privacy for agent i [9, Section 2.3].

In this work, we use the Gaussian mechanism, which adds zero-mean i.i.d. noise drawn from a Gaussian distribution. We define it using the Q -function, i.e., $Q(y) = \frac{1}{\sqrt{2\pi}} \int_y^\infty e^{-\frac{z^2}{2}} dz$.

Lemma 1 (Gaussian Mechanism [16]). *Let $b_i > 0$, $\epsilon_i > 0$, and $\delta_i \in (0, \frac{1}{2})$ be given, and let $x_i \in \ell_\infty^d$. Then the Gaussian mechanism takes the form $\tilde{x}_i(k) = x_i(k) + v_i(k)$, where v_i is a stochastic process with $v_i(k) \sim \mathcal{N}(0, \Xi_{v_i})$, where $\Xi_{v_i} = \sigma_i^2 I_d$ with $\sigma_i \geq \frac{b_i}{2\epsilon_i} (K_{\delta_i} + \sqrt{K_{\delta_i}^2 + 2\epsilon_i})$ and $K_{\delta_i} = Q^{-1}(\delta_i)$. This mechanism provides (ϵ_i, δ_i) -differential privacy to x_i .*

3 Problem Formulation

3.1 Problem Statement

Problem 1. *Let Assumption 1 hold. Let $x_i(k) \in \mathbb{R}^d$ be agent i 's state at time k , N_i be agent i 's neighborhood set, and $\gamma > 0$ be a stepsize. Let $n_i(k)$ denote the process noise in agent i 's state dynamics at time k . We define $\Delta_{ij} \in \mathbb{R}^d$ for all $(i, j) \in E$ as the desired relative state offset between agents i and j . Do each of the following:*

[i.]

1. Implement the formation control protocol

$$x_i(k+1) = x_i(k) + \gamma \sum_{j \in N_i} w_{ij}(x_j(k) - x_i(k) - \Delta_{ij}) + n_i(k), \quad \text{自然干扰} \quad (1)$$

in a differentially private, decentralized manner.

2. Bound the performance of the network in terms of privacy parameters and network properties.

3. Use those bounds to co-design the communication topology and privacy parameters of the network.

Next, we define agent- and network-level dynamics and detail how each agent will enforce differential privacy.

3.2 Multi-Agent Formation Control

The goal of formation control is for agents to reach a set of relative states. Multi-agent formation control is a well-researched problem and there are several mathematical formulations one can use to achieve similar results [27, 28, 29, 30, 31, 32, 33]. We will define offsets between agents that communicate and the objective is for all agents to maintain these relative offsets.

Assumption 2 (Section 6.1.1 of [33]). *The formation specified by $\{\Delta_{ij}\}_{(i,j) \in E}$ is feasible, i.e., there exists a set of points that simultaneously satisfies all pairwise distances. We require $\Delta_{ij} = -\Delta_{ji}$ for all $(i, j) \in E$.*

The objective is having $\lim_{k \rightarrow \infty} (x_j(k) - x_i(k)) = \Delta_{ij}$ for all $(i, j) \in E$. The formation can be centered on any point in $\mathbb{R}^d$, i.e., we allow formations to be translationally invariant [33].

Remark 1. *For all $i \in [N]$, agent i will execute a privatized form of (1) to be detailed below. To ease discussion, we will consider a set of points $\mathcal{P} = \{p_1, \dots, p_N\}$ below that satisfy $p_j - p_i = \Delta_{ij}$ for all $(i, j) \in E$. We emphasize here that agents do not know (and do not need to know) the point p_i for any $i \in [N]$. Instead, our use of $\mathcal{P}$ is only for ease of discussion and analysis. Algorithm 1 below shows the steps that each agent will actually implement, and none of them require p_i .*

As in (1), we model agents as discrete-time single integrators, i.e., $x_i(k+1) = x_i(k) + u_i(k) + n_i(k)$, where $n_i(k) \sim \mathcal{N}(0, s_i^2 I_d)$ is process noise and $u_i(k) \in \mathbb{R}^d$ is agent i 's input. Let the vector $p = (p_1^T, \dots, p_N^T)^T \in \mathbb{R}^{Nd}$ be a network-level state that satisfies the formation specification, let $x(k) = (x_1(k)^T, \dots, x_N(k)^T)^T \in \mathbb{R}^{Nd}$, and let $n(k) = (n_1(k)^T, \dots, n_N(k)^T)^T \in \mathbb{R}^{Nd}$. Set $\bar{x}(k) = x(k) - p$. Then analyzing (1) at the network-level is equivalent to analyzing

$$\bar{x}(k+1) = ((I_N - \gamma L(\mathcal{G})) \otimes I_d) \bar{x}(k) + n(k), \quad (2)$$

where $\otimes$ is the Kronecker product; we reiterate that (2) is used only for analysis, and agents do not need to know p_i for any i .

Let $\bar{x}_{i[l]}$ be the l^{th} scalar element of $\bar{x}_i$ for $l \in [d]$. Then $\bar{x}_{[l]} = [\bar{x}_{1[l]}, \dots, \bar{x}_{N[l]}]^T$ is the vector of all agents' states in the l^{th} dimension, and $\bar{n}_{[l]} = [n_{1[l]}, \dots, n_{N[l]}]^T$ is the vector of corresponding noise terms. Analyzing (2) is therefore equivalent to analyzing the protocol

$$\bar{x}_{[l]}(k+1) = (I_N - \gamma L(\mathcal{G})) \bar{x}_{[l]}(k) + n_{[l]}(k) \quad (3)$$

for all $l \in [d]$ simultaneously.

3.3 Private Communications (Solution to Problem 1.1)

To privately implement the protocol in (1), agent j starts by selecting privacy parameters $\epsilon_j > 0$, $\delta_j \in (0, \frac{1}{2})$, and adjacency relation Adj_{b_j} with $b_j > 0$. Then, agent j privatizes its state trajectory x_j with the Gaussian mechanism. Let $\tilde{x}_j$ denote the differentially private version of x_j , where $\tilde{x}_j(k) = x_j(k) + v_j(k)$, with $v_j(k) \sim \mathcal{N}(0, \Xi_{v_j})$. Here, $\Xi_{v_j} = \sigma_j^2 I_d$ and $\sigma_j \geq \frac{b_j}{2\epsilon_j} (K_{\delta_j} + \sqrt{K_{\delta_j}^2 + 2\epsilon_j})$. Lemma 1 shows that this setup keeps agent j 's state trajectory (ϵ_j, δ_j) -differentially private. Agent j then shares $\tilde{x}_j(k)$ with its neighbors.

Thus, agent i only has access to $\tilde{x}_j(k)$ for $j \in N_i$. Plugging this into the node-level formation control protocol in (1) gives

$$\tilde{x}_i(k+1) = \tilde{x}_i(k) + \gamma \sum_{j \in N_i} w_{ij} (\tilde{x}_j(k) - \tilde{x}_i(k) - \Delta_{ij}) + n_i(k). \quad (4)$$

It is well known that (3) and (4) are mathematically equivalent (see, e.g., [33]). We analyze (3) because it is simpler, and all of these analyses apply verbatim to (4).

We present the full private formation controller in Algorithm 1, which solves Problem 1.1.

Algorithm 1: Private formation controller (Solution to Problem 1.1)

Inputs: $\epsilon_i > 0$, $\delta_i \in (0, 1/2)$, $b_i > 0$ for all $i \in [N]$, Δ_{ij} for all $(i, j) \in E$

Outputs: Privately assembled formation

for $k = 1, 2, \dots$ **do**

for $i \in [N]$ **do**

 Agent i sends $\tilde{x}_i(k)$ to agents $j \in N_i$

 Agent i receives $\tilde{x}_j(k)$ for all $j \in N_i$

 Agent i executes the state update in (4)

We note that the Gaussian input perturbation mechanism employed here is standard. The novelty of this work lies not in developing a new mechanism but instead in the system-level analysis of privacy's impact on multi-agent control performance, as well as the incorporation of this analysis into a computational framework for privacy/network co-design, which we develop across this section and Sections 4 and 5. To solve Problems 1.2 and 1.3, we will analyze this protocol at the network level.

Lemma 2. *Consider Algorithm 1 and let Assumption 1 hold. Then the network-level dynamics for each $l \in \{1, \dots, d\}$ are*

$$\bar{x}_{[l]}(k+1) = (I_N - \gamma L(\mathcal{G}))\bar{x}_{[l]}(k) + z_{[l]}(k),$$

where $z_{[l]}(k) \sim \mathcal{N}(0, \Xi_z)$, $\Xi_z = \gamma^2 A(\mathcal{G})\Xi_v A(\mathcal{G}) + \Xi_n$, $\Xi_v = \text{diag}(\sigma_1^2, \dots, \sigma_N^2)$, and $\Xi_n = \text{diag}(s_1^2, \dots, s_N^2)$.

Proof. See A.1. □

To analyze network-level performance, we define $\beta_{[l]}(k) := \frac{1}{N} \mathbf{1}^T \bar{x}_{[l]}(k) \mathbf{1}$, which is the state vector that the protocol in (3) would converge to with initial state $x_{[l]}(k)$ and without privacy or process noise. Also let

$$e_{[l]}(k) = \bar{x}_{[l]}(k) - \beta_{[l]}(k), \quad (5)$$

which is the offset of the current state from the state the protocol would converge to without noise.

4 Performance of Differentially Private Formation Control

In this section we solve Problem 1.2. First, we compute the total mean square error in a formation. Then, we use these results to derive performance bounds. In the forthcoming proofs, we will analyze each dimension separately for simplicity. However, results will be stated in terms of all dimensions simultaneously because it is that form of network-level performance that is ultimately of interest.

We quantify steady-state performance using the total mean-square error of the network, denoted e_{ss} . Agent i 's private formation control protocol in Algorithm 1 runs in $\mathbb{R}^d$, and each agent in the network runs this protocol. This is equivalent to running d identical copies of (3), which is in $\mathbb{R}^N$. Thus, using (3), we can compute the mean square error in dimension l and then multiply by d to compute e_{ss} . The mean-square error in dimension l is equal to $\lim_{k \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N E[e_{[l],i}^2(k)]$, where $e_{[l],i}(k)$ is the i^{th} element of $e_{[l]}(k)$ in (5). Then we have $e_{ss} := \lim_{k \rightarrow \infty} \frac{d}{N} \sum_{i=1}^N E[e_{[l],i}^2(k)]$.

4.1 Connections with the Lyapunov Equation

The main error bound in this paper uses the fact that we can represent the total error in the system as the trace of a covariance matrix. We define $\Xi_{e[l]}(k) = E[e[l](k)e[l](k)^T]$ and $\Xi_\infty = \lim_{k \rightarrow \infty} \Xi_{e[l]}(k)$; we omit the subscript l since Ξ_∞ is the same in each dimension.

Then we have $e_{ss} = \frac{d}{N} \text{Tr}(\Xi_\infty)$. Now we will analyze the dynamics of $e[l](k)$ and $\Xi_{e[l]}(k)$. For a given $\gamma > 0$ and a given graph $\mathcal{G}$, let $\mathcal{M} = I_N - \gamma L(\mathcal{G}) - \frac{1}{N} \mathbb{1}\mathbb{1}^T$. Then we have the following.

Lemma 3. *Consider Algorithm 1, and let Assumptions 1 and 2 hold. Then the network level error $e[l](k)$ in (5) evolves via*

$$e[l](k+1) = \mathcal{M}e[l](k) + (I_N - \frac{1}{N} \mathbb{1}\mathbb{1}^T)z[l](k), \quad (6)$$

$\Xi_{e[l]}(k)$ evolves according to

$$\Xi_{e[l]}(k+1) = \mathcal{M}\Xi_{e[l]}(k)\mathcal{M} + (I_N - \frac{1}{N} \mathbb{1}\mathbb{1}^T)\Xi_z(I_N - \frac{1}{N} \mathbb{1}\mathbb{1}^T), \quad (7)$$

and $\Xi_{e[l]}(k)$ can be computed via

$$\Xi_{e[l]}(k) = \sum_{i=0}^{k-1} \mathcal{M}^i (I_N - \frac{1}{N} \mathbb{1}\mathbb{1}^T) \Xi_z (I_N - \frac{1}{N} \mathbb{1}\mathbb{1}^T) \mathcal{M}^i. \quad (8)$$

Proof. See A.2. □

The following result computes the spectral radius of a matrix that will be used in our analysis below. A similar bound is in [34], but that work selects a vector-valued gain, while we only vary γ , which is a scalar-valued stepsize.

Lemma 4. *Let N agents communicate over a graph that satisfies Assumption 1 with Laplacian $L(\mathcal{G})$. Then the maximum singular value of $\mathcal{M} = I_N - \gamma L(\mathcal{G}) - \frac{1}{N} \mathbb{1}\mathbb{1}^T$ is $\sigma_{\max}(\mathcal{M}) = 1 - \gamma \lambda_2(L)$.*

Proof. See A.3. □

We prove the existence of the steady-state error covariance matrix Ξ_∞ by showing that it solves a discrete-time Lyapunov equation that has a unique solution.

Theorem 1. *Consider N agents executing Algorithm 1, and let Assumptions 1 and 2 hold. Then Ξ_∞ is the unique solution to the discrete time Lyapunov equation*

$$\Xi_\infty = (I_N - \frac{1}{N} \mathbb{1}\mathbb{1}^T) \Xi_z (I_N - \frac{1}{N} \mathbb{1}\mathbb{1}^T) + \mathcal{M} \Xi_\infty \mathcal{M}, \quad (9)$$

where we have $\Xi_z = \gamma^2 A(\mathcal{G}) \text{diag}(\sigma_1^2, \dots, \sigma_N^2) A(\mathcal{G}) + \Xi_n$ and $\sigma_i^2 = \frac{b_i}{2\epsilon_i} (K_{\delta_i} + \sqrt{K_{\delta_i}^2 + 2\epsilon_i})$ is the variance of privacy noise.

Proof. From Lemma 3, we have $\Xi_\infty = \sum_{i=0}^{\infty} \mathcal{M}^i (I_N - \frac{1}{N} \mathbb{1}\mathbb{1}^T) \Xi_z (I_N - \frac{1}{N} \mathbb{1}\mathbb{1}^T) \mathcal{M}^i$. Let $Q = (I_N - \frac{1}{N} \mathbb{1}\mathbb{1}^T) \Xi_z (I_N - \frac{1}{N} \mathbb{1}\mathbb{1}^T)$. Using $\mathcal{M}^0 = I_N$ and factoring gives $\Xi_\infty = Q + \mathcal{M} (\sum_{i=1}^{\infty} \mathcal{M}^{i-1} Q \mathcal{M}^{i-1}) \mathcal{M} = Q + \mathcal{M} \Xi_\infty \mathcal{M}$. By Lemma 4, $\mathcal{M}$ has eigenvalues strictly less than 1, and by Prop. 2.1 in [35], (9) has a unique, symmetric solution. □

Theorem 1, along with the fact that $e_{ss} = \frac{d}{N} \text{Tr}(\Xi_\infty)$, allows us to solve for e_{ss} . Thus, given a graph $\mathcal{G}$ and privacy parameters $\{(\epsilon_i, \delta_i)\}_{i=1}^N$, we can determine the performance of the network, encoded by e_{ss} before runtime. We will use its value to design a communication topology that allows agents to be as private as possible while satisfying $e_{ss} \leq e_R$, where e_R is the maximum allowable error at steady-state.

4.2 Analytical Bound

Next we derive a scalar bound on e_{ss} in terms of the privacy parameters $\{(\epsilon_i, \delta_i)\}_{i=1}^N$, and properties of the graph $\mathcal{G}$.

Theorem 2. *Let all the conditions from Theorem 1 hold. With $e_{ss} := \lim_{k \rightarrow \infty} \frac{d}{N} \sum_{i=1}^N E \left[e_{[l],i}^2(k) \right]$, we have*

$$e_{ss} \leq \frac{\gamma d \sum_{i=1}^N C_{w_i, d_i} \frac{b_i}{2\epsilon_i} \left(K_{\delta_i} + \sqrt{K_{\delta_i}^2 + 2\epsilon_i} \right) + \frac{N-1}{N\gamma} \sum_{i=1}^N s_i^2}{N\lambda_2(L(\mathcal{G})) (2 - \gamma\lambda_2(L(\mathcal{G})))}, \quad (10)$$

where $C_{w_i, d_i} = \sum_{j=1}^N w_{ij}^2 - \frac{d_i^2}{N}$.

Proof. See B. □

Theorem 2 solves Problem 1.2, and we will use it below to design networks for both privacy and performance. For example, if the network must achieve $e_{ss} \leq e_R$, then this can be achieved by requiring that the bound in (10) is less than e_R . This requirement will impose joint constraints on the communication topology $\mathcal{G}$, the weights in the communication topology w_{ij} , and privacy parameters $\{(\epsilon_i, \delta_i)\}_{i=1}^N$, which we will use in co-design.

5 Privacy and Network Co-design

In this section we solve Problem 1.3. Our goal is to design a network that meets global performance requirements and agent-level privacy requirements. If some agents use strong privacy, then the high-variance privacy noise they use may make global performance poor, even if many other agents have only weak privacy requirements. This effect can also be influenced by the topology of the network, e.g., if an agent with strong privacy sends very noisy messages to other agents along heavily weighted edges. Privacy/network co-design thus requires unifying and balancing these tradeoffs.

This co-design framework is designed for agents that wish to participate in a joint co-design setup. Agent i is able to constrain the value of ϵ_i that is generated by the co-design problem, and this constraint is permitted with the understanding that agent i will use the value of ϵ_i that is computed in solving the co-design problem.

5.1 Co-Design Search Space

We consider N agents that implement private formation control and each agent has a minimum strength of privacy that it will accept. Our objective is to make agents as private as possible while meeting a global performance requirement. In many settings, each agent's neighborhood set will be fixed *a priori* based on hardware compatibility or physical location, and these neighborhoods specify an unweighted, undirected graph of edges that can be used.

We will design the privacy parameters for each agent and all edge weights for the given graph. That is, we can assign a zero or non-zero weight to each edge that is present, but we cannot assign non-zero weight to an edge that is absent. We denote the given unweighted graph of possible edges by $\mathcal{G}_0$ and denote its unweighted Laplacian by L_0 . We define $\mathcal{L}(L_0)$ as the space of all weighted graph Laplacians L such that $L_{ij} = 0$ if $L_{0,ij} = 0$. Co-design will find the optimal weighted Laplacian in $\mathcal{L}(L_0)$ and privacy parameters for each agent.

5.2 Co-Design Problem Statement

Below we derive the privacy/network co-design problem.

Problem 2 (Ideal Co-design). *Given an undirected, unweighted, simple graph $\mathcal{G}_0$ with Laplacian L_0 , a required global error bound e_R , an edge weight budget B , and a minimum level of privacy ϵ_i^{max} for agent i*

for all $i \in [N]$, to co-design privacy and agents' communication topology, solve

$$\begin{aligned}
& \min_{L(\mathcal{G}) \in \mathcal{L}(L_0), \{\epsilon_i\}_{i=1}^N} \sum_{i=1}^N \epsilon_i^2 - \lambda_2(L(\mathcal{G})) \\
& \text{subject to} \\
& \frac{\gamma d \sum_{i=1}^N C_{w_i, d_i} \frac{b_i}{2\epsilon_i} \left(K_{\delta_i} + \sqrt{K_{\delta_i}^2 + 2\epsilon_i} \right) + \frac{N-1}{N\gamma} \sum_{i=1}^N s_i^2}{N\lambda_2(L)(2 - \gamma\lambda_2(L))} \leq e_R \\
& \epsilon_i \leq \epsilon_i^{max} \quad \text{for all } i \text{ and } \text{Tr}(L(\mathcal{G})) \leq 2B.
\end{aligned}$$

In the objective function, the term $\sum_{i=1}^N \epsilon_i^2$, helps make each agent's privacy stronger since privacy is stronger as ϵ_i shrinks. We choose a quadratic function to penalize large privacy parameters more heavily, but this choice is not essential for our framework, and any convex function of the decision variables $\{\epsilon_i\}_{i \in [N]}$ is admissible. For the term $-\lambda_2(L(\mathcal{G}))$, minimizing this helps produce a network which is as connected as possible, i.e., maximizing the algebraic connectivity will produce a solution which promotes better network-level convergence properties. The first constraint requires that the total mean square error of the network is less than some user-defined value e_R . The second constraint requires that each agent's privacy level obeys the constraint that the agent has specified. The last constraint budgets how much edge weight can be used, specifically the total edge weight used may not exceed B .

Overall, solving Problem 2 will produce a communication topology and privacy parameters that will meet a global performance constraint while allowing agents to be as private as possible. This solves Problem 1.3. The main computational challenge in solving Problem 2 comes from the objective function and first constraint. Both depend on the algebraic connectivity of the graph $\mathcal{G}$, namely $\lambda_2(L(\mathcal{G}))$, and for N nodes the computation of $\lambda_2(L(\mathcal{G}))$ has $O(N^3)$ complexity [36]. Furthermore, Problem 2 is nonconvex due to the constraint on steady-state error, and thus one typically cannot expect to find a global solution to it. To address these challenges while maintaining the essential structure of the co-design problem, we develop a relaxed formulation in the following subsection.

5.3 Biconvex Co-Design

We introduce a biconvex relaxation of Problem 2 that has more favorable numerical properties while maintaining the structure of the original problem. We present the relaxed problem first, then discuss its derivation, biconvexity, and solution approach.

Problem 3 (Implemented Co-design). *Given the same inputs as Problem 2, solve*

$$\begin{aligned}
& \min_{L(\mathcal{G}) \in \mathcal{L}(L_0), \{\epsilon_i\}_{i=1}^N, y \in [0, N]} \sum_{i=1}^N \epsilon_i^2 - y \\
& \text{subject to} \\
& \frac{\gamma d \sum_{i=1}^N C_{w_i, d_i} \frac{b_i}{2\epsilon_i} \left(K_{\delta_i} + \sqrt{K_{\delta_i}^2 + 2\epsilon_i} \right) + \frac{N-1}{N\gamma} \sum_{i=1}^N s_i^2}{Ny(2 - \gamma y)} \leq e_R \\
& \epsilon_i \leq \epsilon_i^{max} \quad \text{for all } i \text{ and } \text{Tr}(L(\mathcal{G})) \leq 2B \\
& y \leq \lambda_2(L(\mathcal{G})).
\end{aligned} \tag{11}$$

To derive Problem 3, we decouple the nonconvex constraint involving $\lambda_2(L(\mathcal{G}))$ by introducing an auxiliary variable $y \in [0, N]$. We rewrite the error constraint from Problem 2 as

$$\frac{\gamma d}{Ne_R} \sum_{i=1}^N C_{w_i, d_i} \frac{b_i}{2\epsilon_i} \left(K_{\delta_i} + \sqrt{K_{\delta_i}^2 + 2\epsilon_i} \right) + \frac{N-1}{e_R N^2 \gamma} \sum_{i=1}^N s_i^2 \leq \lambda_2(L)(2 - \gamma\lambda_2(L(\mathcal{G}))),$$

and then split this into two separate constraints: first requiring that the left-hand side be bounded by $y(2 - \gamma y)$, and second requiring that $y \leq \lambda_2(L)$. The problem still requires careful handling due to the

product of C_{w_i, d_i} and the function of ϵ_i in the first constraint. This product is what creates the biconvex structure that we analyze next.

Definition 5 (Biconvex program [37]). *Let $X \subseteq \mathbb{R}^m$ and $Y \subseteq \mathbb{R}^n$ be convex sets, and let $(x, y) \in X \times Y$. A function $f : X \times Y \rightarrow \mathbb{R}$ is biconvex if, for every fixed $y \in Y$, the function $x \mapsto f(x, y)$ is convex on X , and for every fixed $x \in X$, the function $y \mapsto f(x, y)$ is convex on Y . A constraint set $S \subseteq X \times Y$ is biconvex if, for every fixed $y \in Y$, the set $\{x \in X \mid (x, y) \in S\}$ is convex, and for every fixed $x \in X$, the set $\{y \in Y \mid (x, y) \in S\}$ is convex. An optimization problem of the form*

$$\min_{(x, y) \in X \times Y} f(x, y) \quad \text{subject to } (x, y) \in S$$

is called a biconvex program if f and S are biconvex.

This framework requires partitioning the decision variables of Problem 3 into two disjoint sets, and we partition them into $(L(\mathcal{G}_0), y)$ and $(\{\epsilon_i\}_{i=1}^N)$. The following theorem establishes that Problem 3 is biconvex.

Theorem 3. *Given an undirected, simple, weighted graph $\mathcal{G}_0$, a global error requirement e_R , an edge budget B , and a minimum level of privacy ϵ_i^{max} for agent i for all $i \in [N]$, Problem 3 is a biconvex program with respect the variables $(L(\mathcal{G}_0), y)$ and $\{\epsilon_i\}_{i=1}^N$.*

Proof. The objective function (11) is positive definite and quadratic as a function of $\{\epsilon_i\}_{i=1}^N$ and is thus convex in the privacy parameters. The objective is linear in y . This implies that the objective is biconvex in $(L(\mathcal{G}_0), y)$ and $\{\epsilon_i\}_{i=1}^N$.

To analyze the error constraint

$$\frac{\gamma d}{N e_R} \sum_{i=1}^N C_{w_i, d_i} \frac{b_i}{2\epsilon_i} \left(K_{\delta_i} + \sqrt{K_{\delta_i}^2 + 2\epsilon_i} \right) + \frac{N-1}{e_R N^2} \sum_{i=1}^N s_i^2 - y(2 - \gamma y) \leq 0, \quad (12)$$

recall that $C_{w_i, d_i} = \sum_{j=1}^N w_{ij}^2 - \frac{d_i^2}{N}$ and $d_i = \sum_{j \mid (i, j) \in E} w_{ij}$. For $i \in [N]$, let $w_i = [w_{i1}, \dots, w_{iN}]^T \in \mathbb{R}^N$. Then

$$\begin{aligned} C_{w_i, d_i} &= \sum_{j=1}^N w_{ij}^2 - \frac{d_i^2}{N} = w_i^T w_i - \frac{1}{N} (\mathbf{1}^T w_i)^2 \\ &= w_i^T w_i - \frac{1}{N} w_i^T \mathbf{1} \mathbf{1}^T w_i = w_i^T \left(I_N - \frac{1}{N} \mathbf{1} \mathbf{1}^T \right) w_i. \end{aligned}$$

Note that $I_N - \frac{1}{N} \mathbf{1} \mathbf{1}^T \succeq 0$ and thus C_{w_i, d_i} is convex for all $i \in [N]$. It can be shown that $\frac{b_i}{2\epsilon_i} (K_{\delta_i} + \sqrt{K_{\delta_i}^2 + 2\epsilon_i})$ is a convex, nonincreasing function of ϵ_i for any $b_i > 0$ and $\delta_i \in (0, 1/2)$. Lastly, $-y(2 - \gamma y)$ is convex in y . Therefore, for fixed y and w_{ij} , the error constraint (12) is convex in $\{\epsilon_i\}_{i=1}^N$. Similarly, for fixed $\{\epsilon_i\}_{i=1}^N$, the error constraint (12) is convex in y and w_{ij} . Thus (12) is biconvex in $(L(\mathcal{G}_0), y)$ and $\{\epsilon_i\}_{i=1}^N$.

The algebraic connectivity $\lambda_2(L(\mathcal{G}))$ can be expressed as $\lambda_2(L(\mathcal{G})) = \min_{x \text{ s.t. } \|x\|=1, x^T \mathbf{1}=0} x^T L(\mathcal{G}) x$. This is an infimum over a set of linear maps and is therefore concave as a function of edge weights, and the constraint $y \leq \lambda_2(L(\mathcal{G}))$ is convex. Lastly, the constraint $\text{Tr}(L(\mathcal{G})) \leq 2B$ is a linear function of edge weights and is therefore convex. Overall, this shows that Problem 3 meets the definition of a biconvex program from Definition 5. $\square$

Having established that Problem 3 is biconvex, we now consider approaches for its solution. A global optimization algorithm for biconvex programs exists [37], but it guarantees global optimality by iteratively partitioning the feasible region and maintaining associated convex relaxations and bounds for each subregion of the partition. As the number of subregions grows, this method must store a growing collection of subproblems and bounding information, which can lead to exponential space complexity in the worst case. Instead, we employ an alternate convex search approach [38, Algorithm 4.1] that maintains a single iterate and alternates between two convex subproblems. At each iteration, only the current decision variables and fixed problem data are stored, and thus the space complexity scales as $O(|E| + N)$. Specifically, our algorithm alternates between: (1) solving for optimal edge weights $\{w_{ij}\}_{i, j \in [N]}$ and auxiliary variable y while fixing

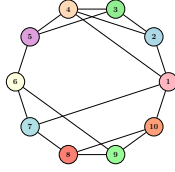


Figure 1: The graph used for simulations. Edge weights are not specified and will be optimized over.

privacy parameters $\{\epsilon_i\}_{i \in [N]}$, and (2) solving for optimal privacy parameters $\{\epsilon_i\}_{i \in [N]}$ while fixing weights and y .

This approach has several desirable properties. First, each subproblem is convex and can be solved efficiently using standard convex optimization tools. The wide availability of software for solving optimization problems therefore makes it simple to deploy this framework in practice. Second, the alternating scheme is guaranteed to converge to a stationary point since the objective function is bounded below and either decreases or remains constant at each iteration [38]. We deliberately avoid adding any more assumptions that could strengthen our convergence analysis because we want this co-design framework to have the broadest reach possible. Third, while global optimality is not analytically guaranteed, our numerical results in the following section demonstrate that our solutions yield high-quality co-designs that successfully meet both privacy and formation requirements in practice. In fact, as we will show, computational co-design results often exceed the minimum privacy requirements, exceed the minimum performance requirements, or exceed both, indicating strong levels of performance in practice. The usefulness of these stationary points follows from the analytical performance characterizations developed earlier, which ensure that any feasible stationary solution satisfies the desired privacy and steady-state performance guarantees. Strong performance numerically thus follows from the preceding analytical developments that have been made.

6 Simulations

In this section we provide simulation results for optimal privacy/network co-design¹. This section does not provide any numerical comparisons to prior work because we are not aware of any comparable prior work that co-designs a network topology and a trajectory-level differential privacy implementation. We first define an input topology as the undirected, unweighted, simple graph $\mathcal{G}$ shown in Figure 1. Then, we run privacy/network co-design to obtain edge weights and privacy parameters $\{\epsilon_i\}_{i=1}^N$. In these results, the smaller a node is drawn the more private it is, i.e., ϵ_i gets smaller as node i shrinks, and the thicker an edge is drawn the more weight it has. In simulation, edges with weights less than 10^{-4} are considered deleted. Within each example, the same solution was reached regardless of the initial iterate used by the optimization algorithm.

Example 1: (Trading off privacy and performance) Fix the input graph $\mathcal{G}$ shown in Figure 1. Fix $\epsilon^{max} = [0.4, 0.9, 0.55, 0.35, 0.8, 0.45, 0.7, 0.5, 0.52, 0.58]^T$, $B = 6$, $\gamma = 1/2N$, $\delta_i = 0.05$, and $b_i = 1$ for all i . Now let e_R take on the values $e_R \in \{8, 16, 64\}$. For each of these values, privacy/network co-design was used to design the weighted graphs shown in Figure 2 and the privacy parameters shown in Figure 3.

In Figure 2, as we allow weaker performance, quantified by larger e_R , the agents are able to use a much stronger level of privacy. This is illustrated by the nodes shrinking as e_R increases from Figure 2a to 2c. The edge weights are roughly constant due to the fixed budget of $B = 6$ and the biconvex nature of the problem. In Figure 3, we also see that as the required level of performance decreases, co-design allows the agents to be more private. This trend persists for all agents in varying magnitudes, which are influenced by the topology and each ϵ_i^{max} . Moreover, Figure 3 shows that, even under the tightest constraint on e_{ss} , agents implement values of ϵ_i ranging from approximately 0.04 to 0.07, which are in line with some of the strongest privacy protections that have been used in the literature [26]. Thus, even when using input perturbation for privacy, there is an inherent compatibility between privacy and formation control, and agents can simultaneously attain both high performance and strong privacy protections. $\triangle$

¹All code is available on GitHub [39].

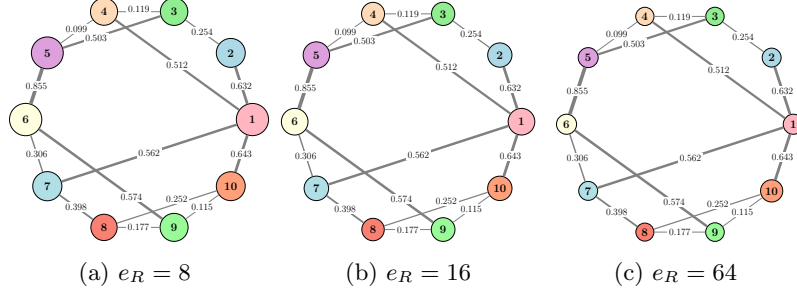


Figure 2: Graphs generated in Example 1 for $e_R \in \{8, 16, 64\}$. The smaller a node is drawn, the smaller its value of ϵ_i is. The thicker an edge is drawn, the larger its weight is. When we allow weaker performance, indicated by larger e_R , each agent has a stronger level of privacy and the topology uses less weight as illustrated by the nodes shrinking from Figure 2a to Figure 2c.



Figure 3: The privacy parameters designed in Example 1 with $e_R \in \{8, 16, 64\}$.

Example 2: (Trading off edge budget and privacy) We fix all the parameters as given in Example 1, however, we set $e_R = 16.0$ and let B take on the values $B \in \{2, 4, 8\}$, i.e., we fix a level of performance and modulate the edge weight budget. The co-designed edge weights and privacy parameters are given in Figures 4 and 5.

In Figure 4, as the budget is increased more edge weight is used and the agents get *less* private. As B is increased more edge weight is used to increase the $\lambda_2(L(\mathcal{G}))$ term of the objective. However, high edge weights amplify the privacy noise injected by agents, which has a negative impact on steady state error. Therefore, to meet the performance requirement of $e_R = 16$, co-design is making agents less private when we produce networks with high edge weights. This is directly controlled by B . The numerical values of the privacy parameters are presented in Figure 5, where we observe the privacy parameters roughly doubling between $B = 2$ and $B = 8$.

Remark 2. *These results show that the proposed privacy and network co-design framework effectively handles the increased amount of noise required by input perturbation by incorporating the privacy noise in the system analysis and design.*

7 Conclusions

We have developed a framework to design networks and privacy parameters for differentially private formation control, which allows agents to privately assemble formations with bounded steady-state error. This framework was developed for cooperative agents, and future work includes quantifying the extent to which it is robust to non-cooperative and adversarial agents. Such agents may inject false information into the network (among other attacks) and a promising direction for future work is to explore how privacy's insensitivity to changes in one agent's state trajectory can help make such attacks less effective.

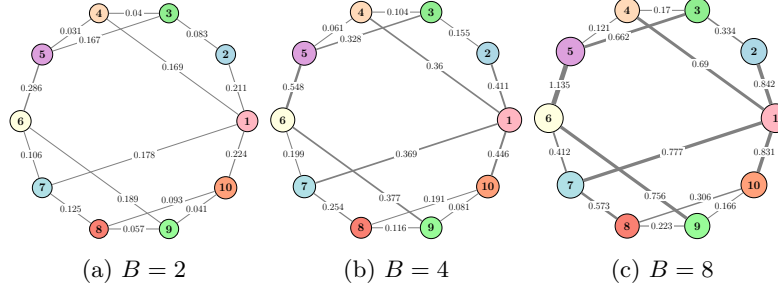


Figure 4: The outputs of privacy/network co-design with fixed parameters specified in Example 2 for different values of $B \in \{2, 4, 8\}$.



Figure 5: The privacy parameters from Example 2 with $B \in \{2, 4, 8\}$.

A Proofs of Supporting Lemmas

A.1 Proof of Lemma 2

From (4), we subtract p_i from both sides to find

$$\bar{x}_i(k+1) = \bar{x}_i(k) + \gamma \sum_{j \in N_i} w_{ij}(\bar{x}_j(k) + v_j(k) - \bar{x}_i(k)) + n_i(k),$$

which we factor as

$$\bar{x}_i(k+1) = \bar{x}_i(k) + \gamma \sum_{j \in N_i} w_{ij}(\bar{x}_j(k) - \bar{x}_i(k)) + z_i(k),$$

where $z_i(k) = \gamma \sum_{j \in N_i} w_{ij}v_j(k) + n_i(k)$. Note that $z_i(k) = \gamma([A(\mathcal{G})]_{\text{row } i} \otimes I_d)v(k) + n_i(k)$, where $[A(\mathcal{G})]_{\text{row } i} \in \mathbb{R}^{1 \times N}$ is the i^{th} row of $A(\mathcal{G})$ and $\otimes$ is the Kronecker product. Next we define the vector

$$z_{[l]}(k) = [z_{1[l]}(k)^T, \dots, z_{N[l]}(k)^T]^T \in \mathbb{R}^N$$

giving $z_{[l]}(k) = \gamma A(\mathcal{G})v_{[l]}(k) + n_{[l]}(k)$ at the network level. The covariance of $z_{[l]}(k)$ can then be calculated as $\Xi_z = E[(\gamma A(\mathcal{G})v_{[l]}(k) + n_{[l]}(k))(\gamma A(\mathcal{G})v_{[l]}(k) + n_{[l]}(k))^T]$. Since $v_{[l]}(k)$ and $n_{[l]}(k)$ are statistically independent, $E[v_{[l]}(k)n_{[l]}(k)^T] = E[n_{[l]}(k)v_{[l]}(k)^T] = 0$. Applying this and the linearity of expectation gives $\Xi_z = \gamma^2 A(\mathcal{G})\Xi_v A(\mathcal{G}) + \Xi_n$. Then $z_{[l]}(k) \sim \mathcal{N}(0, \gamma^2 A(\mathcal{G})\Xi_v A(\mathcal{G}) + \Xi_n)$. $\square$

A.2 Proof of Lemma 3

Expanding the error term $e_{[l]}$ gives

$e_{[l]}(k+1) = (I_N - \gamma L(\mathcal{G}) - \frac{1}{N}\mathbb{1}\mathbb{1}^T)\bar{x}_{[l]}(k) + (I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)z_{[l]}(k)$, where we have used the fact that $L(\mathcal{G})\mathbb{1} = 0$. Let $\mathcal{M} = I_N - \gamma L(\mathcal{G}) - \frac{1}{N}\mathbb{1}\mathbb{1}^T$ and note that we also have $\mathcal{M}\beta_{[l]}(k) = (I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)\beta_{[l]}(k) = 0$.

Then we have the equality $e_{[l]}(k+1) = \mathcal{M}e_{[l]}(k) + (I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)z_{[l]}(k)$, which proves (6). Plugging this into the definition of $\Xi_{e_{[l]}}(k+1)$ gives $\Xi_{e_{[l]}}(k+1) = \mathcal{M}\Xi_{e_{[l]}}(k)\mathcal{M} + (I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)\Xi_z(I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)$, which proves (7). Then (8) follows by applying (7) recursively. $\square$

A.3 Proof of Lemma 4

We begin by analyzing the eigenvalues of $\mathcal{M}$. Note that $\mathcal{M}\mathbb{1} = 0$, and thus $\mathcal{M}$ has eigenvalue 0 with eigenvector $\mathbb{1}$. Now let (λ, v) be an eigenpair of $\mathcal{M}$ with $\lambda \neq 0$. We will show that $\lambda \neq 0$ implies that (λ, v) is also an eigenpair of $I - \gamma L(\mathcal{G})$. Since $\mathcal{G}$ is connected and undirected, $I - \gamma L(\mathcal{G})$ is a doubly stochastic matrix. Thus, it has one eigenvalue of modulus 1 with an eigenvector of $\mathbb{1}$, and the other eigenvalues are positive and strictly inside the unit disk [32, Lemma 3].

Using $\mathbb{1}^T(I_N - \gamma L(\mathcal{G})) = \mathbb{1}^T$, we have $\mathbb{1}^T v = \mathbb{1}^T(I_N - \gamma L(\mathcal{G}))v$. Adding and subtracting $\mathbb{1}^T(\frac{1}{N}\mathbb{1}\mathbb{1}^T)v$ gives $\mathbb{1}^T v = \mathbb{1}^T \mathcal{M}v + \mathbb{1}^T(\frac{1}{N}\mathbb{1}\mathbb{1}^T)v$, then plugging in $\mathcal{M}v = \lambda v$ and $\mathbb{1}^T(\frac{1}{N}\mathbb{1}\mathbb{1}^T) = \mathbb{1}^T$ gives $\mathbb{1}^T v = \lambda \mathbb{1}^T v + \mathbb{1}^T v = (\lambda + 1)\mathbb{1}^T v$. Since $\lambda \neq 0$, we must have $\mathbb{1}^T v = 0$, and thus v is orthogonal to $\mathbb{1}$. Furthermore we have that $(\frac{1}{N}\mathbb{1}\mathbb{1}^T)v = 0$, so $\lambda v = \mathcal{M}v = (I_N - \gamma L(\mathcal{G}) - \frac{1}{N}\mathbb{1}\mathbb{1}^T)v = (I_N - \gamma L(\mathcal{G}))v$. Any non-zero eigenvalue of $\mathcal{M}$ is also an eigenvalue of $I - \gamma L(\mathcal{G})$ and the eigenvector v is orthogonal to $\mathbb{1}$.

Furthermore, $L(\mathcal{G})$ and $I_N - \gamma L(\mathcal{G})$ have the same eigenvectors and $\lambda_i(I_N - \gamma L(\mathcal{G})) = 1 - \gamma\lambda_i(L(\mathcal{G}))$. Thus, $\lambda_i(\mathcal{M}) = 1 - \gamma\lambda_i(L(\mathcal{G}))$ for $i \in \{2, \dots, N\}$, and we have $\lambda_{\max}(\mathcal{M}) = 1 - \gamma\lambda_2(L(\mathcal{G})) < 1$. Thus, all eigenvalues of $\mathcal{M}$ lie strictly in the unit disk. Now $\sigma_{\max}(\mathcal{M}) = 1 - \gamma\lambda_2(L)$ follows from the fact that $[\sigma_{\max}(\mathcal{M})]^2 = [(\lambda_{\max}((\mathcal{M})(\mathcal{M})))^{1/2}]^2 = \lambda_{\max}((\mathcal{M}))^2 = (1 - \gamma\lambda_2(L(\mathcal{G})))^2$. $\square$

B Proof of Theorem 2

With Theorem 1 and [40, Equation (151)] we have

$$Tr(\Xi_\infty) \leq \frac{Tr(Q)}{1 - (\sigma_{\max}(\mathcal{M}))^2}, \quad (13)$$

where $\sigma_{\max}(\mathcal{M})$ denotes the maximum singular value of $\mathcal{M}$.

We next expand $Tr(Q)$ in (13) and apply the cyclic permutation property of the trace to find

$$Tr(Q) = Tr\left((I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)(I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)\Xi_z\right). \quad (14)$$

Note that $(I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)(I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T) = I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T$. Plugging this and $\Xi_z = \gamma^2 A(G)\Xi_v A(G) + \Xi_n$ into (14) gives

$$Tr(Q) = \gamma^2 Tr\left((I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)A(G)\Xi_v A(G)\right) + Tr\left((I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)\Xi_n\right). \quad (15)$$

With $\Xi_n = \text{diag}(s_1^2, \dots, s_N^2)$, the i^{th} diagonal term of $(I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)\Xi_n$ is given by $[(I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)\Xi_n]_{ii} = (1 - \frac{1}{N})s_i^2$. Then we have that the last term in (15) is

$$Tr\left((I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)\Xi_n\right) = \frac{N-1}{N} \sum_{i=1}^N s_i^2. \quad (16)$$

To simplify $\gamma^2 Tr((I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)A(G)\Xi_v A(G))$, cyclic permutation of the trace gives that

$$\gamma^2 Tr\left((I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)A(G)\Xi_v A(G)\right) = \gamma^2 Tr\left(A(G)(I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)A(G)\Xi_v\right).$$

The i^{th} diagonal term of $A(G)(I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)A(G)$ is $[A(G)(I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)A(G)]_{ii} = \sum_{j=1}^N w_{ij}^2 - \frac{d_i^2}{N}$, and

$$\gamma^2 Tr\left(A(G)(I_N - \frac{1}{N}\mathbb{1}\mathbb{1}^T)A(G)\Xi_v\right) = \gamma^2 \sum_{i=1}^N C_{w_i, d_i} \sigma_i^2. \quad (17)$$

Using (16)-(17) in (15) gives $Tr(Q) = \gamma^2 \sum_{i=1}^N C_{w_i, d_i} \sigma_i^2 + \frac{N-1}{N} \sum_{i=1}^N s_i^2$, and plugging this into (13) gives $Tr(\Xi_\infty) \leq \frac{\gamma^2 \sum_{i=1}^N C_{w_i, d_i} \sigma_i^2 + \frac{N-1}{N} \sum_{i=1}^N s_i^2}{1 - (\sigma_{\max}(\mathcal{M}))^2}$. Using Lemma 4 to expand $(\sigma_{\max}(\mathcal{M}))^2$ and then using $e_{ss} = \frac{d}{N} Tr(\Xi_\infty)$ completes the proof. $\square$

References

- [1] C. Dwork, F. McSherry, K. Nissim, A. Smith, Calibrating noise to sensitivity in private data analysis, in: Theory of cryptography conference, Springer, 2006, pp. 265–284.
- [2] Y. Wang, Z. Huang, S. Mitra, G. E. Dullerud, Differential privacy in linear distributed control systems: Entropy minimizing mechanisms and performance tradeoffs, IEEE Transactions on Control of Network Systems 4 (1) (2017) 118–130.
- [3] K. Yazdani, A. Jones, K. Leahy, M. Hale, Differentially private lq control, arXiv preprint arXiv:1807.05082 (2018).
- [4] Y. Kawano, M. Cao, Design of privacy-preserving dynamic controllers, IEEE Transactions on Automatic Control 65 (9) (2020) 3863–3878.
- [5] V. Krishnan, S. Martínez, A probabilistic framework for moving-horizon estimation: Stability and privacy guarantees, IEEE Transactions on Automatic Control 66 (4) (2020) 1817–1824.
- [6] S. Han, U. Topcu, G. J. Pappas, Differentially private distributed constrained optimization, IEEE Transactions on Automatic Control 62 (1) (2016) 50–64.
- [7] Z. Huang, S. Mitra, N. Vaidya, Differentially private distributed optimization, in: Proceedings of the 2015 international conference on distributed computing and networking, 2015, pp. 1–10.
- [8] Y. Wang, M. Hale, M. Egerstedt, G. Dullerud, Differentially private objective functions in distributed cloud-based optimization, in: 55th IEEE Conference on Decision and Control (CDC), 2016, pp. 3688–3694.
- [9] C. Dwork, A. Roth, The algorithmic foundations of differential privacy, Foundations and Trends® in Theoretical Computer Science 9 (3–4) (2014) 211–407.
- [10] S. P. Kasiviswanathan, A. Smith, On the ‘semantics’ of differential privacy: A bayesian formulation, Journal of Privacy and Confidentiality 6 (1) (2014).
- [11] J. Chen, J. Wang, J.-F. Zhang, Differentially private distributed nonconvex stochastic optimization with quantized communication, IEEE Transactions on Automatic Control (2025).
- [12] J. Zhang, J. Lu, J. Liang, J. Zhong, Average consensus of whole-process privacy protection: A scale parameter method, Information Fusion 107 (2024) 102312.
- [13] Z. Huang, S. Mitra, G. Dullerud, Differentially private iterative synchronous consensus, in: Proceedings of the 2012 ACM Workshop on Privacy in the Electronic Society, New York, NY, USA, 2012, p. 81–90.
- [14] E. Nozari, P. Tallapragada, J. Cortés, Differentially private average consensus: Obstructions, trade-offs, and optimal algorithm design, Automatica 81 (2017) 221–231.
- [15] M. Ruan, H. Gao, Y. Wang, Secure and privacy-preserving consensus, IEEE Transactions on Automatic Control 64 (10) (2019) 4035–4049.
- [16] J. Le Ny, G. J. Pappas, Differentially private filtering, IEEE Transactions on Automatic Control 59 (2) (2013) 341–354.
- [17] C. Gao, Z. Wang, L. Ma, H. Liu, X. He, Private synchronization control of complex networks against eavesdropping attacks: A differential privacy scheme, IEEE Transactions on Network Science and Engineering 11 (3) (2024) 3028–3038.

- [18] Z. Lu, G. Guo, Control and communication scheduling co-design for networked control systems: A survey, *International Journal of Systems Science* 54 (1) (2023) 189–203.
- [19] J. Tan, J. Wang, J. Zhang, Cooperative secure parameter identification of multi-participant arx systems—a threshold paillier cryptosystem-based least-squares identification algorithm, *Scientia Sinica Informationis* 53 (12) (2023) 2472–2492.
- [20] L. Guo, J. Wang, Y. Zhao, J.-F. Zhang, State estimation with protecting exogenous inputs via cramer-rao lower bound approach, *arXiv preprint arXiv:2410.08756* (2025).
- [21] C. Hawkins, M. Hale, Differentially private formation control, in: 59th IEEE Conference on Decision and Control (CDC), 2020, pp. 6260–6265.
- [22] M. Fiedler, Algebraic connectivity of graphs, *Czechoslovak mathematical journal* 23 (2) (1973) 298–305.
- [23] C. Dwork, Differential privacy, *Automata, languages and programming* (2006) 1–12.
- [24] K. Yazdani, M. Hale, Error bounds and guidelines for privacy calibration in differentially private kalman filtering, in: 2020 American Control Conference (ACC), 2020, pp. 4423–4428.
- [25] B. Hajek, *Random processes for engineers*, Cambridge university press, 2015.
- [26] J. Hsu, M. Gaboardi, A. Haeberlen, S. Khanna, A. Narayan, B. C. Pierce, A. Roth, Differential privacy: An economic method for choosing epsilon, in: 2014 IEEE 27th Computer Security Foundations Symposium, IEEE, 2014, pp. 398–410.
- [27] A. Jadbabaie, A. Olshevsky, Scaling laws for consensus protocols subject to noise (2015). **arXiv:1508.00036**.
- [28] L. Krick, M. E. Broucke, B. A. Francis, Stabilisation of infinitesimally rigid formations of multi-robot networks, *International Journal of control* 82 (3) (2009) 423–439.
- [29] W. Ren, R. W. Beard, E. M. Atkins, Information consensus in multivehicle cooperative control, *IEEE Control systems magazine* 27 (2) (2007) 71–82.
- [30] W. Ren, Consensus strategies for cooperative control of vehicle formations, *IET Control Theory & Applications* 1 (2) (2007) 505–512.
- [31] J. A. Fax, R. M. Murray, Information flow and cooperative control of vehicle formations, *IEEE transactions on automatic control* 49 (9) (2004) 1465–1476.
- [32] R. Olfati-Saber, J. A. Fax, R. M. Murray, Consensus and cooperation in networked multi-agent systems, *Proceedings of the IEEE* 95 (1) (2007) 215–233.
- [33] M. Mesbahi, M. Egerstedt, *Graph theoretic methods in multiagent networks*, Princeton University Press, 2010.
- [34] K. You, L. Xie, Network topology and communication data rate for consensusability of discrete-time multi-agent systems, *IEEE Transactions on Automatic Control* 56 (10) (2011) 2262–2275.
- [35] P. M. Gahinet, A. J. Laub, C. S. Kenney, G. Hower, Sensitivity of the stable discrete-time lyapunov equation, *IEEE Transactions on Automatic Control* 35 (11) (1990) 1209–1217.
- [36] B. N. Parlett, The qr algorithm, *Computing in science & engineering* 2 (1) (2000) 38–42.
- [37] C. A. Floudas, V. Visweswaran, A global optimization algorithm (gop) for certain classes of nonconvex nlp—i. theory, *Computers & chemical engineering* 14 (12) (1990) 1397–1417.
- [38] J. Gorski, F. Pfeuffer, K. Klamroth, Biconvex sets and optimization with biconvex functions: a survey and extensions, *Mathematical methods of operations research* 66 (3) (2007) 373–407.

- [39] C. Hawkins, Privacy-network-co-design, <https://github.com/fx0641/Differentially-Private-Formation-Control-Privacy-Network-Co-Design> (2025).
- [40] W. H. Kwon, Y. S. Moon, S. C. Ahn, Bounds in algebraic riccati and lyapunov equations: a survey and some new results, *International Journal of Control* 64 (3) (1996) 377–389.